

Cleaning up the trivial relocation APIs in C++26

Document #: P3631R0 [[Latest](#)] [[Status](#)]
Date: 2025-05-13
Project: Programming Language C++
Audience: LEWG, LWG
Reply-to: Louis Dionne
<ldionne.2@gmail.com>
Giuseppe D'Angelo
<giuseppe.dangelo@kdab.com>

Contents

- [1 Introduction](#)
- [2 Proposal](#)
- [3 Wording changes](#)
 - [3.1 Synopsis of relocation functions 20.2.2 \[memory.syn\]](#)
 - [3.2 Definitions for relocation functions \(20.2.6 \[obj.lifetime\]\)](#)

§ 1 Introduction

[P3516](#) was design-approved by LEWG in Hagenberg. As part of that discussion, we also discussed cleaning up the relocation-related APIs provided in the library (since [P2786](#) also added some). There was a fair amount of consensus in the room that the introduction of [P3516](#) removed the need for some of the APIs added by [P2786](#). However, we decided to punt it to a separate paper to allow [P3516](#) to progress. This is the cleanup paper.

§ 2 Proposal

The current state of the APIs that have been voted into the draft or design approved in Hagenberg is this:

```
// P2786 APIs (in draft)
template <class T>
T* trivially_relocate(T* first, T* last, T* result); // freestanding

template <class T>
constexpr T* relocate(T* first, T* last, T* result); // freestanding
```

```

// P3516 APIs (design approved)
template<class T>
    requires relocatable-from<T, T>
    constexpr T* relocate_at(T* dest, T* source)
        noexcept(is_nothrow_relocatable_v<T>);

template<class NoThrowForwardIterator1, class NoThrowForwardIterator2>
    constexpr NoThrowForwardIterator2
        uninitialized_relocate(NoThrowForwardIterator1 first,
                               NoThrowForwardIterator1 last,
                               NoThrowForwardIterator2 result); //
    freestanding

template<class NoThrowForwardIterator1, class Size, class
    NoThrowForwardIterator2>
    constexpr pair<NoThrowForwardIterator1, NoThrowForwardIterator2>
        uninitialized_relocate_n(NoThrowForwardIterator1 first,
                                 Size n,
                                 NoThrowForwardIterator2 result); //
    freestanding

template<class NoThrowBidirectionalIterator1, class
    NoThrowBidirectionalIterator2>
    constexpr NoThrowBidirectionalIterator2
        uninitialized_relocate_backward(NoThrowBidirectionalIterator1 first,
                                         NoThrowBidirectionalIterator1 last,
                                         NoThrowBidirectionalIterator2 result);
    // freestanding

// ... also std::execution_policy overloads ...

namespace ranges {
    // ... similar overloads here ...
}

```

To clean things up, we propose removing `std::relocate(...)`. It was introduced before we had proper relocation algorithms, and it's basically a less capable version of the uninitialized algorithms above since it doesn't support iterators and potentially-throwing move constructors. One difference is that `std::relocate(...)` does detect overlaps and decides to relocate forward or backward, which P3516 handles by having a `_backward` variant instead (which is consistent with existing uninitialized algorithms).

Note that we *do not* propose removing `std::trivially_relocate` or making it an exposition-only helper, although the idea was explored. While we don't want to encourage users to call it, we decided that the function had a useful role as a low level object manipulation function, similarly to `std::memmove`. This will be a simple wrapper over the compiler builtin that all compilers are expected to provide, which may be helpful to write some low level APIs portably. General users are still expected to use the higher level `std::uninitialized_xxx` algorithms instead.

§ 3 Wording changes

§ 3.1 Synopsis of relocation functions 20.2.2 [memory.syn]

Remove `std::relocate` from the synopsis:

```
template <class T>
    constexpr T* relocate(T* first, T* last, T* result);
    // freestanding
```

§ 3.2 Definitions for relocation functions (20.2.6 [obj.lifetime])

Remove `std::relocate`. In *Explicit lifetime management* 20.2.6 [obj.lifetime]:

```
template <class T>
    constexpr T* relocate(T* first, T* last, T* result);
```

Mandates: `is_nothrow_relocatable_v<T> && !is_const_v<T>` is true. T is not an array of unknown bound.

Preconditions:

- `[first, last)` is a valid range.
- `[result, result + (last - first))` denotes a region of storage that is a subset of the region reachable through `result` (6.8.4 [basic.compound]) and suitably aligned for the type T.
- No element in the range `[first, last)` is a potentially-overlapping subobject.

Effects:

- If `result == first` is true, no effect;
- otherwise, if not called during constant evaluation and `is_trivially_relocatable_v<T>` is true, then has effects equivalent to: `trivially_relocate(first, last, result);`
- otherwise, for each integer `i` in `[0, last - first)`,
 - if T is an array type, equivalent to: `relocate(begin(first[i]), end(first[i]), *start_lifetime_as<T>(result + i));`
 - otherwise, equivalent to: `construct_at(result + i, std::move(first[i])); destroy_at(first + i);`

Returns: `result + (last-first)`.

Throws: Nothing.

[Note: Overlapping ranges are supported. — end note]